

Phase 4: Heapify — Faster Heap Construction

1. Current Progress

You have already completed the first three phases of the heap curriculum.

What you learned in Phase 1

You learned the basic foundation of heaps:

- A heap is a special binary tree.
- A heap must be a **complete binary tree**.
- A heap is usually stored in an array.
- In this curriculum, index `0` is ignored.
- The root starts at index `1`.
- For a node at index `i`:
 - Left child = $2 * i$
 - Right child = $2 * i + 1$
 - Parent = $i / 2$
- A max heap keeps the largest value at the root.
- A min heap keeps the smallest value at the root.
- A heap is not fully sorted.
- A heap only guarantees correct parent-child order.

What you learned in Phase 2

You learned insertion.

The main idea of insertion is:

Insert at the next free position first, then shift up.

Insertion moves:

Bottom → Top
Leaf → Root

During insertion, we compare the inserted value with its **parent**.

For a max heap:

If child > parent, the child moves upward.

What you learned in Phase 3

You learned deletion and heap sort.

The main idea of deletion is:

Delete the root, move the last element to the root, then shift down.

Deletion moves:

Top → Bottom
Root → Leaf

During deletion, we compare the moved value with its **children**.

For a max heap:

Choose the larger child during shift-down.

What Phase 4 teaches

Phase 4 teaches **heapify**.

Heapify is a faster way to create a heap from an unsorted array.

The most important memory rule before starting Phase 4 is:

Insertion moves upward.
Deletion moves downward.
Heapify also moves downward.

Heapify becomes easy now because you already understand **shift-down** from deletion.

2. What This Phase Covers

In this phase, you will learn:

1. What heapify means.
 2. Why heapify is used.
 3. The old heap creation method using repeated insertion.
 4. Why repeated insertion takes $O(N \log N)$.
 5. Why heapify is faster.
 6. The direction of heapify.
 7. Why heapify uses shift-down.
 8. Why leaves are skipped.
 9. Why heapify starts at $n / 2$.
 10. What an internal node is.
 11. What a leaf node is.
 12. How the `shiftDown` helper function works.
 13. How the `heapify` function works.
 14. A full dry run of heapify.
 15. The complete C program for heapify.
 16. Time complexity of heapify.
 17. Space complexity of heapify.
 18. Common beginner mistakes.
 19. Important rules to memorize.
 20. Practice questions and answers.
-

3. Big Idea of Phase 4

In Phase 2, you learned one way to create a heap:

Insert elements one by one.

That method works, but it is not the fastest way.

In Phase 4, you learn a better method:

Heapify

Heapify takes the whole unsorted array and rearranges it into a heap more efficiently.

Simple meaning:

Heapify = turn an unsorted array into a heap faster.

4. What Is Heapify?

Heapify means converting an unsorted array into a valid heap.

Example unsorted array:

5, 10, 30, 20, 35, 40, 15

After heapify, it can become this max heap:

40, 35, 30, 20, 10, 5, 15

This final array is a valid max heap because every parent is greater than or equal to its children.

Important:

Heapify does not sort the array.
Heapify only creates a heap.

A heap only needs this rule:

Parent $\geq$ Child for max heap
Parent $\leq$ Child for min heap

In this phase, we mainly focus on **max heap heapify**.

5. Heapify Does Not Mean Sorting

Many beginners think heapify should fully sort the array.

That is wrong.

Heapify only makes the array satisfy the heap property.

Example final heap:

40, 35, 30, 20, 10, 5, 15

This is not fully sorted in descending order because 15 comes after 5.

But it is still a valid max heap.

Why?

Because the parent-child relationships are correct:

```
40 >= 35 and 40 >= 30
35 >= 20 and 35 >= 10
30 >= 5 and 30 >= 15
```

So remember:

Heapify creates a heap, not a sorted array.

6. Old Method: Create Heap by Repeated Insertion

In Phase 2, you created a heap by repeated insertion.

The code looked like this:

```
for (int i = 2; i <= n; i++) {
    insert(A, i);
}
```

The idea was:

1. Treat `A[1]` as a one-element heap.
2. Insert `A[2]`.
3. Insert `A[3]`.
4. Continue until `A[n]`.

Each insertion uses shift-up.

So the method works from left to right:

Index 2 → Index 3 → Index 4 → ... → Index n

Complexity of repeated insertion

One insertion takes:

$O(\log N)$

If we insert N elements, total time becomes:

$O(N \log N)$

This is correct, but heapify is faster.

7. New Method: Heapify

Heapify does not insert elements one by one.

Instead, it treats the entire array as a complete binary tree first.

Then it fixes the tree from bottom to top.

Heapify works like this:

Start from the last internal node.
Move backward toward the root.
Apply shiftDown at each internal node.

So heapify moves:

Bottom internal nodes → Root

And each node adjustment moves:

Downward toward the leaves

8. Repeated Insertion vs Heapify

Feature	Repeated Insertion	Heapify
Direction through array	Left to right	Right to left
Starting point	Index <code>2</code>	Index <code>n / 2</code>
Ending point	Index <code>n</code>	Index <code>1</code>
Movement used	Shift-up	Shift-down
Time complexity	<code>O(N log N)</code>	<code>O(N)</code>
Main idea	Grow heap one element at a time	Fix subtrees bottom-up

Simple memory:

Repeated insertion builds upward.
Heapify fixes downward.

9. Direction Comparison

Operation	Starts From	Moves Toward	Compares With
Insertion	New leaf	Root	Parent
Deletion	Root	Leaf	Children
Heapify	Internal node	Leaf	Children

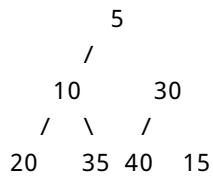
Important memory:

Insertion = shift up
Deletion = shift down
Heapify = shift down

10. What Is a Leaf Node?

A leaf node is a node with no children.

Example tree:



The leaf nodes are:

20, 35, 40, 15

They have no children.

In array form:

Index:	1	2	3	4	5	6	7
Value:	5	10	30	20	35	40	15

The leaf indexes are:

4, 5, 6, 7

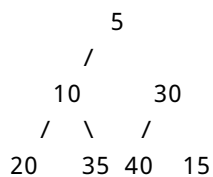
A leaf is already a valid heap by itself because it has no children to violate the heap rule.

So heapify skips leaves.

11. What Is an Internal Node?

An internal node is a node that has at least one child.

In this tree:



The internal nodes are:

5, 10, 30

In array form:

Index:	1	2	3	4	5	6	7
Value:	5	10	30	20	35	40	15

The internal node indexes are:

1, 2, 3

These are the nodes that may need fixing.

Heapify starts at the last internal node.

12. Why Does Heapify Start at $n / 2$?

In a 1-based heap array, the formulas are:

```
Left child  = 2 * i
Right child = 2 * i + 1
Parent      = i / 2
```

For a heap with n elements, indexes greater than $n / 2$ are leaves.

That means:

Indexes from $n / 2 + 1$ to n are leaves.

Leaves have no children, so they do not need shift-down.

Therefore, heapify starts at:

$n / 2$

This is the last internal node.

Example

If:

$$n = 7$$

Then:

$$n / 2 = 7 / 2 = 3$$

So heapify starts at index **3**.

Indexes:

1, 2, 3, 4, 5, 6, 7

Internal nodes:

1, 2, 3

Leaves:

4, 5, 6, 7

So heapify processes:

3 → 2 → 1

13. Why Heapify Moves Right to Left

Heapify starts near the bottom and moves toward the root.

It processes internal nodes from right to left:

$$n / 2, n / 2 - 1, n / 2 - 2, \dots, 1$$

For **n = 7**, this is:

3, 2, 1

Why?

Because before fixing a parent, we want its child subtrees to already be valid heaps.

Heapify works bottom-up.

Simple meaning:

Fix small lower subtrees first.
Then fix bigger upper subtrees.
Finally fix the root.

14. What Is `shiftDown` ?

`shiftDown` is a helper function.

It fixes one subtree by moving a value downward.

For a max heap, `shiftDown` works like this:

1. Start at a parent index `i`.
2. Find its left child using `2 * i`.
3. Find its right child using `2 * i + 1`.
4. Choose the larger child.
5. If the parent is smaller than the larger child, swap them.
6. Continue checking downward.
7. Stop when the parent is greater than or equal to its children.

Simple meaning:

If a small value is above a bigger child, push it downward.

15. Why We Choose the Larger Child in a Max Heap

In a max heap, the bigger value must stay higher.

Suppose we have:

```
      5
     /
    35   40
```

The children are:

35 and 40

The larger child is:

40

Because this is a max heap, 40 should move above 5.

If we swapped 5 with 35, the heap would still be broken because 40 would still be below 5.

So the rule is:

In max heap shift-down, always choose the larger child.

For a min heap, the opposite is true:

In min heap shift-down, choose the smaller child.

16. shiftDown Function in C

```
void shiftDown(int A[], int i, int n) {
    int j = 2 * i; // left child

    while (j <= n) {
        // Choose the larger child
        if (j < n && A[j + 1] > A[j]) {
            j = j + 1;
        }

        // If parent is smaller than larger child, swap
        if (A[i] < A[j]) {
            int temp = A[i];
            A[i] = A[j];
            A[j] = temp;
        }
    }
}
```

```
        A[j] = temp;

        i = j;
        j = 2 * j;
    } else {
        break;
    }
}
```

17. Line-by-Line Explanation of `shiftDown`

Function header

```
void shiftDown(int A[], int i, int n)
```

This function shifts the value at index `i` downward until the subtree becomes a valid max heap.

Here:

Variable	Meaning
<code>A[]</code>	Heap array
<code>i</code>	Current parent index being checked
<code>n</code>	Number of active heap elements

Find the left child

```
int j = 2 * i;
```

In a 1-based heap array:

```
Left child = 2 * i
```

So `j` first points to the left child.

Continue while a child exists

```
while (j <= n)
```

This means:

Continue while the current node has at least a left child.

If `j > n`, there is no child, so we stop.

Choose the larger child

```
if (j < n && A[j + 1] > A[j]) {  
    j = j + 1;  
}
```

This checks whether the right child exists and whether it is larger than the left child.

`j < n` means the right child exists.

Why?

Because right child is at:

```
j + 1
```

If `j` is already equal to `n`, then there is no right child.

If the right child is larger, we move `j` to the right child.

After this step:

`j` points to the larger child.

Compare parent with larger child

```
if (A[i] < A[j])
```

This checks if the parent is smaller than the larger child.

For a max heap, this is wrong:

```
Parent should be >= Child
```

So if `A[i] < A[j]`, we need to swap.

Swap parent and child

```
int temp = A[i];  
A[i] = A[j];  
A[j] = temp;
```

This swaps the parent with the larger child.

The larger child moves upward.

The smaller parent moves downward.

Move to the next level

```
i = j;  
j = 2 * j;
```

After swapping, the smaller value is now at index `j`.

So we update:

```
i = j
```

Now we continue shift-down from the new position.

Then:

```
j = 2 * j
```

This finds the left child of the new position.

Stop if heap property is correct

```
else {  
    break;  
}
```

If the parent is already greater than or equal to the larger child, then the subtree is valid.

So we stop.

18. Heapify Function in C

```
void heapify(int A[], int n) {  
    for (int i = n / 2; i >= 1; i--) {  
        shiftDown(A, i, n);  
    }  
}
```

19. Line-by-Line Explanation of `heapify`

Function header

```
void heapify(int A[], int n)
```

This function converts the array into a heap.

Here:

Variable	Meaning
<code>A[]</code>	Array to be turned into a heap
<code>n</code>	Number of active elements

Start at the last internal node

```
for (int i = n / 2; i >= 1; i--)
```

This loop starts at `n / 2`.

Why?

Because `n / 2` is the last internal node.

Indexes after `n / 2` are leaves, and leaves do not need fixing.

Move backward to the root

```
i >= 1
```

This means the loop continues until index `1`, the root.

Apply shiftDown

```
shiftDown(A, i, n);
```

This fixes the subtree rooted at index `i`.

So heapify fixes:

```
Last internal subtree  
Then previous internal subtree  
Then previous internal subtree  
Finally the root subtree
```

20. Full Dry Run of Heapify

We will heapify this array:

Index:	1	2	3	4	5	6	7
Value:	5	10	30	20	35	40	15

Tree view:

```
      5
     /
    10   30
   / \  /
  20 35 40 15
```

This is not a max heap because:

5 is smaller than 10 and 30.
10 is smaller than 20 and 35.
30 is smaller than 40.

Now we apply heapify.

Since:

$n = 7$

We start at:

$n / 2 = 7 / 2 = 3$

So we process:

$i = 3$
 $i = 2$
 $i = 1$

21. Dry Run Step 1: Heapify Index 3

Current index:

$i = 3$

Current value:

$A[3] = 30$

Children of index **3**:

Left child index = $2 * 3 = 6$
Right child index = $2 * 3 + 1 = 7$

Values:

$A[6] = 40$
 $A[7] = 15$

Larger child:

40

Compare parent with larger child:

$30 < 40$

This breaks the max heap rule.

So swap **30** and **40**.

Array becomes:

5, 10, 40, 20, 35, 30, 15

Tree becomes:

```
      5
     /
    10  40
```

```
  /  \  /
20  35 30 15
```

Now the subtree rooted at index **3** is fixed.

22. Dry Run Step 2: Heapify Index 2

Current index:

`i = 2`

Current value:

`A[2] = 10`

Children of index **2**:

Left child index = $2 * 2 = 4$
Right child index = $2 * 2 + 1 = 5$

Values:

`A[4] = 20`
`A[5] = 35`

Larger child:

35

Compare parent with larger child:

`10 < 35`

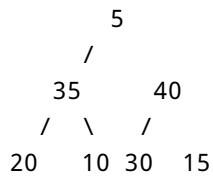
This breaks the max heap rule.

So swap **10** and **35**.

Array becomes:

5, 35, 40, 20, 10, 30, 15

Tree becomes:



Now the subtree rooted at index 2 is fixed.

23. Dry Run Step 3: Heapify Index 1

Current index:

$i = 1$

Current value:

$A[1] = 5$

Children of index 1:

Left child index = $2 * 1 = 2$
Right child index = $2 * 1 + 1 = 3$

Values:

$A[2] = 35$
 $A[3] = 40$

Larger child:

40

Compare parent with larger child:

5 < 40

This breaks the max heap rule.

So swap 5 and 40.

Array becomes:

40, 35, 5, 20, 10, 30, 15

Tree becomes:

```
      40
     /
    35   5
   / \  /
  20 10 30 15
```

But we are not done yet.

The value 5 moved down to index 3.

Now we must continue shifting it down.

Children of index 3:

Left child index = $2 * 3 = 6$
Right child index = $2 * 3 + 1 = 7$

Values:

A[6] = 30
A[7] = 15

Larger child:

30

Compare:

5 < 30

So swap 5 and 30.

Array becomes:

40, 35, 30, 20, 10, 5, 15

Tree becomes:

```
      40
     /
    35  30
   / \ /
  20 10 5 15
```

Now heapify is complete.

24. Final Heap Check

Final array:

Index:	1	2	3	4	5	6	7
Value:	40	35	30	20	10	5	15

Check every parent-child relationship.

Index 1

A[1] = 40
A[2] = 35
A[3] = 30

Check:

40 >= 35
40 >= 30

Valid.

Index 2

```
A[2] = 35  
A[4] = 20  
A[5] = 10
```

Check:

```
35 >= 20  
35 >= 10
```

Valid.

Index 3

```
A[3] = 30  
A[6] = 5  
A[7] = 15
```

Check:

```
30 >= 5  
30 >= 15
```

Valid.

Therefore:

```
40, 35, 30, 20, 10, 5, 15
```

is a valid max heap.

25. Complete Phase 4 C Program

```
#include <stdio.h>  
  
void shiftDown(int A[], int i, int n) {  
    int j = 2 * i;
```



```

while (j <= n) {
    if (j < n && A[j + 1] > A[j]) {
        j = j + 1;
    }

    if (A[i] < A[j]) {
        int temp = A[i];
        A[i] = A[j];
        A[j] = temp;

        i = j;
        j = 2 * j;
    } else {
        break;
    }
}

}

void heapify(int A[], int n) {
    for (int i = n / 2; i >= 1; i--) {
        shiftDown(A, i, n);
    }
}

void printArray(int A[], int n) {
    for (int i = 1; i <= n; i++) {
        printf("%d ", A[i]);
    }
    printf("\n");
}

int main() {
    // Index 0 is ignored.
    int A[] = {0, 5, 10, 30, 20, 35, 40, 15};
    int n = 7;

    printf("Before heapify: ");
    printArray(A, n);

    heapify(A, n);

    printf("After heapify: ");
    printArray(A, n);

    return 0;
}

```

26. Expected Output

```
Before heapify: 5 10 30 20 35 40 15  
After heapify: 40 35 30 20 10 5 15
```

The first line is the unsorted array.

The second line is the max heap after heapify.

Important:

The output is a heap, not a sorted array.

27. Important Variables in the Program

Variable	Meaning
<code>A[]</code>	The array being converted into a heap
<code>n</code>	Number of active heap elements
<code>i</code>	Current internal node being heapified
<code>j</code>	Child index used during shift-down
<code>temp</code>	Temporary variable used for swapping
<code>A[1]</code>	Root of the heap
<code>A[j]</code>	Larger child during max heap shift-down

28. Why Heapify Is $O(N)$

At first, some learners think heapify should be:

$O(N \log N)$

because `shiftDown` can take $O(\log N)$.

But heapify is actually:

$O(N)$

Why?

Because not every node moves down the full height.

Most nodes are near the bottom.

Many nodes are leaves and do not move at all.

Nodes just above the leaves can move only one level.

Only the root can move the full height.

So the total work across all nodes becomes linear:

$O(N)$

Simple meaning:

Heapify is fast because most nodes do very little work.

29. Complexity Summary

Operation	Time Complexity	Reason
One <code>shiftDown</code>	$O(\log N)$	A value may move down the height of the heap
Heap creation by repeated insertion	$O(N \log N)$	N insertions, each up to $\log N$
Heapify	$O(N)$	Most nodes are near the bottom and move little or not at all
Extra space	$O(1)$	The same array is rearranged

30. Space Complexity

Heapify uses the same array.

It does not create another array.

It only uses a few variables:

```
i, j, temp
```

So the extra space complexity is:

```
O(1)
```

This means heapify is an in-place method.

31. Max Heap Heapify vs Min Heap Heapify

This phase focuses on max heap heapify.

For a max heap:

```
Parent >= Child
```

During shift-down:

```
Choose the larger child.
```

For a min heap:

```
Parent <= Child
```

During shift-down:

```
Choose the smaller child.
```

Comparison table

Feature	Max Heap	Min Heap
Root contains	Largest value	Smallest value

Feature	Max Heap	Min Heap
Parent-child rule	Parent $\geq$ Child	Parent $\leq$ Child
During shift-down choose	Larger child	Smaller child
Used when	Highest value has priority	Lowest value has priority

32. Common Beginner Mistakes

Mistake 1: Starting heapify at `n`

Wrong idea:

Start heapify from the last index.

Why wrong?

The last index is a leaf.

Leaves do not need fixing.

Correct idea:

Start from $n / 2$.

Mistake 2: Calling `shiftDown` on every node

Wrong idea:

Run `shiftDown` from index `n` down to 1.

Why wrong?

Indexes after `$n / 2$` are leaves.

Calling `shiftDown` on leaves wastes time.

Correct idea:

Run shiftDown from $n / 2$ down to 1.

Mistake 3: Using shift-up inside heapify

Wrong idea:

Heapify should use insertion logic.

Why wrong?

Insertion uses shift-up.

Heapify uses deletion-style shift-down.

Correct idea:

Heapify = shiftDown from bottom internal nodes to root.

Mistake 4: Running heapify left to right

Wrong idea:

Process index 1, then 2, then 3.

Why wrong?

Heapify is bottom-up.

Correct idea:

Process $n / 2$, then $n / 2 - 1$, ..., then 1.

Mistake 5: Forgetting to choose the larger child

Wrong idea:

Always swap with the left child.

Why wrong?

In a max heap, the larger child must move upward.

Correct idea:

Compare left and right child first.
Choose the larger child.
Then compare with the parent.

Mistake 6: Thinking heapify sorts the array

Wrong idea:

After heapify, the array should be sorted.

Why wrong?

Heapify only creates heap order.

Correct idea:

Heapify creates a heap, not a sorted array.

Sorting still needs heap sort.

Mistake 7: Thinking heapify is $O(N \log N)$

Wrong idea:

Each `shiftDown` is $O(\log N)$, so heapify must be $O(N \log N)$.

Why wrong?

Most nodes do not move very far.

Many nodes are leaves and are skipped.

Correct idea:

Heapify is $O(N)$.

33. Key Terms

Term	Simple Meaning
Heapify	Turning an unsorted array into a heap
Shift-down	Moving a value downward to restore heap order
Internal node	A node that has at least one child
Leaf node	A node with no children
Last internal node	The node at index $n / 2$
Max heap	Heap where parent is greater than or equal to children
Min heap	Heap where parent is less than or equal to children
Larger child	The bigger child, chosen during max heap shift-down
Bottom-up	Starting from lower nodes and moving toward the root
In-place	Done inside the same array without using another array

34. Important Rules to Memorize

Rule 1

Heapify turns an unsorted array into a heap.

Rule 2

Heapify does not sort the array.

Rule 3

Heapify uses shift-down.

Rule 4

Heapify skips leaves.

Rule 5

Heapify starts at:

$$n / 2$$

Rule 6

Heapify moves backward:

$$n / 2 \rightarrow n / 2 - 1 \rightarrow \dots \rightarrow 1$$

Rule 7

In max heap shift-down, choose the larger child.

Rule 8

Heapify takes:

$$O(N)$$

Rule 9

Heapify uses:

$$O(1)$$

extra space.

35. Simple Practice Questions

Use this array:

Index:	1	2	3	4	5	6	7
Value:	5	10	30	20	35	40	15

Questions:

1. What does heapify do?
2. Does heapify sort the array?
3. What is the value of n ?
4. What is $n / 2$?
5. Which index does heapify start from?
6. Which indexes are leaves?
7. Which indexes are internal nodes?
8. What direction does heapify move through the array?
9. Does heapify use shift-up or shift-down?
10. In a max heap, do we choose the larger child or smaller child during shift-down?
11. What is the time complexity of heapify?
12. What is the extra space complexity of heapify?
13. After heapify, what value should be at the root?
14. Why is heapify faster than repeated insertion?
15. What is the final heap from this example?

36. Practice Answers

1. What does heapify do?

Heapify turns an unsorted array into a heap.

Answer:

It rearranges the array into a valid heap.

2. Does heapify sort the array?

No.

Heapify only creates heap order.

Answer:

No, heapify creates a heap, not a sorted array.

3. What is the value of n ?

There are 7 active elements.

Answer:

$$n = 7$$

4. What is $n / 2$?

$$7 / 2 = 3$$

Integer division is used.

Answer:

3

5. Which index does heapify start from?

Heapify starts at $n / 2$.

Answer:

Index 3

6. Which indexes are leaves?

Indexes after $n / 2$ are leaves.

Since $n / 2 = 3$, leaves are:

4, 5, 6, 7

Answer:

Indexes 4, 5, 6, and 7

7. Which indexes are internal nodes?

Internal nodes are from index 1 to index $n / 2$.

Answer:

Indexes 1, 2, and 3

8. What direction does heapify move through the array?

Heapify moves from the last internal node back to the root.

Answer:

Right to left, from $n / 2$ down to 1.

9. Does heapify use shift-up or shift-down?

Heapify uses deletion-style movement.

Answer:

Shift-down

10. In a max heap, do we choose the larger child or smaller child during shift-down?

In a max heap, the larger value should move upward.

Answer:

Choose the larger child.

11. What is the time complexity of heapify?

Answer:

$O(N)$

12. What is the extra space complexity of heapify?

Heapify rearranges the same array.

Answer:

$O(1)$

13. After heapify, what value should be at the root?

For a max heap, the largest value should be at the root.

The largest value is .

Answer:

40

14. Why is heapify faster than repeated insertion?

Repeated insertion may move many elements upward.

Heapify skips leaves and most nodes move only a short distance.

Answer:

Because most nodes are near the bottom and do little work, so total time is $O(N)$.

15. What is the final heap from this example?

Answer:

40, 35, 30, 20, 10, 5, 15

37. Final Phase 4 Summary

Phase 4 teaches heapify.

Heapify is a faster way to build a heap from an unsorted array.

The old method was:

Repeated insertion = $O(N \log N)$

The new method is:

Heapify = $O(N)$

Heapify works by:

Start at $n / 2$.
Move backward to 1.
Apply shiftDown at each internal node.

Heapify skips leaves because leaves have no children and are already valid heaps.

For a max heap, shiftDown always chooses the larger child.

The final result is a heap, not a sorted array.

Most important beginner memory:

Insertion moves upward.
Deletion moves downward.
Heapify moves downward from bottom internal nodes to the root.

You are now ready for Phase 5: Priority Queues and Full C Implementation.